



University of Frontier Technology, Bangladesh

Department of Cyber Security Engineering

Bachelor of Science (Engg.) in Cyber Security Engineering

Semester: 2024-25 (L1 - T2)

Course title: Object Oriented Programming Language Sessional

Course Code: PROG 112

**Project Report
01**

Project Title: ByteScavenger | File Recovery System

Submitted by:

Sujoy Devnath (20)

MD Zahed Hossen (19)

Department of Cyber
Security Engineering,
University of Frontier
Technology, Bangladesh

Submitted to:

Md. Abdullah

Lecturer

Department of Cyber Security
Engineering,
University of Frontier
Technology, Bangladesh

1. Project Overview

ByteScavenger is a specialized digital forensics tool developed in C++ that recovers "permanently" deleted files from storage media (like USB pendrives). Unlike standard recovery software that relies on the File System's index, this project performs File Carving. It treats the disk as a raw stream of bytes, searching for unique hexadecimal "Magic Bytes" to reconstruct files even when the metadata is destroyed.

2. Problem Statement

When a user "permanently" deletes a file (Shift + Delete) or empties the Recycle Bin, the Operating System does not erase the data. It only marks the space as "available" and removes the file's entry from the File Allocation Table (FAT/NTFS). To the OS, the file is gone. However, to a forensic investigator, the data stays on the physical sectors until it is overwritten. Existing professional tools are often expensive and complicated. There is a need for a simple, OOP-based tool that can handle raw disk acquisition and recovery.

3. Proposed Solution & Objectives

The project proposes a modular C++ application that interacts directly with the hardware layer to recover data.

- ❖ **Objective 1:** Implement a system to automatically discover drives using the Windows API.
- ❖ **Objective 2:** Create a bit-for-bit forensic image (.dd file) to keep the original evidence intact.
- ❖ **Objective 3:** Develop a scanning engine that can identify several file formats (JPEG, PDF, PNG) at the same time.
- ❖ **Objective 4:** Make sure the application can handle system-level security constraints (UAC Elevation) automatically.

4. Target Audience

- **Forensic Students:** As a learning tool for understanding disk structures.
- **Cybersecurity Professionals:** For quick, scriptable data acquisition.
- **System Administrators:** To recover accidentally deleted critical logs or documents from external media.

5. Key Features & Project Scope

- **Self-Elevation:** The program can elevate itself to get the permissions it needs to access the sectors of a disk.
- **Dynamic Drive Selection:** It can find all the physical disks that are connected to the computer and show how much space they have.
- **Forensic Imaging:** The program makes a copy of the physical drive and saves it as a raw binary file so the data on the drive does not get damaged.
- **Signature-Based Carving:** The program looks for files by checking the headers and footers of the files like the code 0xFFD8 that is used for JPEG files.
- **Scope:** The project is about working with files that are not broken into pieces on storage devices like flash drives and hard drives that use FAT32 or exFAT or NTFS. It does not work with encrypted volumes, like BitLocker.

6. Functional & Non-Functional Requirements

Requirements For The System:

- The system should list all physical drives currently connected to the PC.
- The system should copy 100% of the disk sectors into a destination file.
- The system should allow users to choose which file types to recover.

Non-Functional Requirements For The System:

- **Performance:** The carving engine must process data at a rate of at least 50MB/s.
- **Reliability:** The program must handle "Device Unplugged" errors without crashing.
- **Security:** Data must be recovered to a different drive than the source to prevent overwriting.

7. Technology Stack

- **Language:** C++17/20
- **Environment:** Visual Studio and GCC (MinGW).
- **APIs:** Win32 API (for CreateFileA, DeviceIoControl, and ShellExecuteEx).
- **Libraries:** <fstream> for binary I/O, <memory> for smart pointers, <vector> for data buffering.

- **OOP functionalities:** Abstract classes and interfaces, inheritance, Run-time Polymorphism, Encapsulation, Smart Pointers.

8. System Architecture

The system follows a **Manager-Worker** pattern.

- **DeviceManager:** Detects and lists hardware.
- **ForensicImager:** Handles the bit-stream transfer.
- **CarverEngine:** Uses **Polymorphism** to iterate through a list of file signatures and extract matching data.

9. Project Timeline & Milestones

Week	Phase	Key Tasks & Activities
Week 1	System Utilities	Admin elevation and Drive Enumeration
Week 2	Acquisition Engine	Completing the bit-for-bit cloning class.
Week 3	Analysis Engine	Implementing the Signature base class and JPEG/PDF subclasses.
Week 4	Integration & Testing	Testing recovery on real drives and refining the UI.

10. Expected Deliverables

- **Source Code:** Fully commented C++ file.
- **Executable:** Compiled .exe ready for Windows deployment.
- **Project Documentation:** Explaining the OOP design patterns (Inheritance, Encapsulation) used.
- **Test Report:** A demonstration of a successfully recovered file from a disk.